# 03 â€” Diagrams extraction

Exhaustive extraction of every architecture diagram for the **BobaLink** browser-extension implementation. Source: `/Volumes/T7/Projects/Boba/docs/research/browser_extension/Browser Torrent Extension Guide/diagrams/`. Project name in diagrams = **BobaLink**.

## Files read

| # | Diagram | Source formats read | Notes |
|---|---|---|---|
| 01 | System architecture | `.mmd` + `.drawio.xml` | drawio adds Chrome APIs panel, Docker networks, legend flows |
| 02 | Data flow â€” torrent detection â†' download | `.mmd` | (no drawio/ puml) |
| 03 | Data flow â€” tab-groups batch | `.mmd` + `.drawio.xml` | drawio is a stub (swimlanes only) |
| 04 | Sequence â€” single torrent send | `.mmd` + `.puml` | mmd has `par`, puml has `alt` (same two paths) |
| 05 | Sequence â€” tab-group batch send | `.mmd` + `.puml` | |
| 06 | Component diagram | `.mmd` + `.puml` | mmd lists per-file functions; puml is package-only |
| 07 | ER diagram (DATA MODEL) | `.mmd` + `.puml` + `.drawio.xml` | mmd is authoritative (most fields); drawio drops a few fields |
| 08 | Extension lifecycle | `.mmd` | |
| 09 | Offline-queue state machine | `.mmd` | |
| 10 | Security architecture | `.mmd` + `.drawio.xml` | |
| 11 | Build pipeline | `.mmd` | |
| 12 | Compatibility matrix | `.html` (read in full) | no `.mmd`; only `.html`/`.svg` exist |

`.svg` / rendered `.html` exports of 01â€“11 deliberately NOT read (redundant renders). `12-compatibility-matrix.html` read because no text-source `.mmd` exists for it.

# 1) Manifest

See â€œFiles readâ€ table above.

---

# 2) COMPLETE DATA MODEL (ER diagram #07)

6 entities. The `.mmd` is authoritative (carries the most fields + key/comment annotations). PlantUML and drawio render fewer fields (gaps noted per-entity). Types below: `.mmd` uses generic (`string`/`number`/`boolean`/`datetime`/`json`); `.puml`/`.drawio` give SQL types (`VARCHAR`/`TEXT`/`INTEGER`/`BIGINT`/`BOOLEAN`/`DATETIME`/`JSON`).

### Entity: `extension_config` (#07 â€” single per-install extension settings)

| Field | Type (mmd / sql) | Key | Comment |
|---|---|---|---|
| config_id | string / VARCHAR | **PK** | PRIMARY KEY |
| server_url | string / VARCHAR | | Boba server URL |
| api_key | string / VARCHAR | | API authentication key |
| qbittorrent_url | string / VARCHAR | | qBitTorrent WebUI URL |
| username | string / VARCHAR | | qBitTorrent username |
| password_encrypted | string / VARCHAR | | Encrypted password |
| auto_detect | boolean / BOOLEAN | | Auto-detect torrents |
| default_category | string / VARCHAR | | Default qBit category |
| scan_interval_ms | number / INTEGER | | DOM scan interval |
| max_retries | number / INTEGER | | Max retry attempts |
| retry_delay_ms | number / INTEGER | | Base retry delay |
| created_at | datetime / DATETIME | | Record creation time |
| updated_at | datetime / DATETIME | | Last update time |

### Entity: `discovered_torrents` (#07 â€” torrents found on pages)

| Field | Type | Key | Comment |
|---|---|---|---|
| torrent_id | string / VARCHAR | **PK** | UUID |
| infohash | string / VARCHAR | **UK** (unique) | Unique infohash |
| magnet_uri | string / TEXT | | Full magnet URI |
| name | string / VARCHAR | | Torrent display name |
| size_bytes | number / BIGINT | | Torrent size |
| source_url | string / VARCHAR | | Page where found |
| source_tab_id | string / VARCHAR | | Browser tab ID |
| status | string / VARCHAR | | `pending` \| `sent` \| `failed` |
| discovered_at | datetime / DATETIME | | Discovery timestamp |
| sent_at | datetime / DATETIME | | Send timestamp |
| error_message | string / TEXT | | Error if failed |

drawio omits `source_tab_id`. mmd is the superset.

### Entity: `download_queue` (#07 â€” offline/retry queue)

| Field | Type | Key | Comment |
|---|---|---|---|
| queue_id | string / VARCHAR | **PK** | UUID |
| torrent_id | string / VARCHAR | **FK** â†’ discovered_torrents | References discovered |
| magnet_uri | string / TEXT | | Magnet link |
| priority | number / INTEGER | | Queue priority 0â€“10 |
| retry_count | number / INTEGER | | Current retry count |
| status | string / VARCHAR | | `queued` \| `retrying` \| `failed` \| `completed` |
| queued_at | datetime / DATETIME | | Queue entry time |
| last_retry_at | datetime / DATETIME | | Last retry timestamp |
| error_log | string / TEXT | | JSON error history |

drawio omits `last_retry_at`. mmd is the superset.

### Entity: `server_config` (#07 â€” one row per Boba server, multi-server support)

| Field | Type | Key | Comment |
|---|---|---|---|
| server_id | string / VARCHAR | **PK** | UUID |
| name | string / VARCHAR | | Server display name |
| fastapi_url | string / VARCHAR | | FastAPI endpoint URL |
| go_service_url | string / VARCHAR | | Go service endpoint URL |
| qbittorrent_url | string / VARCHAR | | qBitTorrent WebUI URL |
| api_key | string / VARCHAR | | Authentication key |
| is_active | boolean / BOOLEAN | | Is this server active |
| health_check_interval | number / INTEGER | | Health check ms |
| last_health_check | datetime / DATETIME | | Last check timestamp |
| last_check_result | boolean / BOOLEAN | | Last check passed |
| created_at | datetime / DATETIME | | Creation time |

drawio omits `last_health_check`. mmd is the superset.

### Entity: `send_history` (#07 â€” audit log of every send attempt)

| Field | Type | Key | Comment |
|---|---|---|---|
| history_id | string / VARCHAR | **PK** | UUID |
| infohash | string / VARCHAR | | Torrent infohash |
| magnet_uri | string / TEXT | | Magnet link |
| name | string / VARCHAR | | Torrent name |
| server_id | string / VARCHAR | **FK** â†’ server_config | References server_config |
| success | boolean / BOOLEAN | | Send succeeded |

| Field | Type | Key | Comment |
|---|---|---|---|
| qbittorrent_hash | string / VARCHAR | | Hash from qBit |
| category | string / VARCHAR | | qBit category used |
| tags | string / VARCHAR | | qBit tags |
| sent_at | datetime / DATETIME | | Send timestamp |
| response_time_ms | number / INTEGER | | API response time |
| error_message | string / TEXT | | Error if failed |

drawio omits `magnet_uri`, `qbittorrent_hash`, `tags`. mmd is the superset.

### Entity: `search_cache` (#07 â€" cached search results, TTLâ€™d)

| Field | Type | Key | Comment |
|---|---|---|---|
| cache_id | string / VARCHAR | **PK** | UUID |
| query_hash | string / VARCHAR | | Hash of search query |
| query_text | string / VARCHAR | | Original search text |
| results | json / JSON | | Cached results JSON |
| result_count | number / INTEGER | | Number of results |
| cached_at | datetime / DATETIME | | Cache timestamp |
| ttl_seconds | number / INTEGER | | Cache TTL |
| is_valid | boolean / BOOLEAN | | Cache still valid |

`search_cache` has NO declared relationship to any other entity (standalone).

### Relationships (cardinality) (#07)

- `extension_config ||--o{ discovered_torrents` â€" one-to-many â€" label â€œreferences configâ€�
- `discovered_torrents ||--o{ download_queue` â€" one-to-many â€" label â€œqueued for retryâ€� (FK `download_queue.torrent_id`)
- `server_config ||--o{ send_history` â€" one-to-many â€" label â€œsends trackedâ€� (FK `send_history.server_id`)

(`||--o{` = one mandatory â†"ï¸Ž zero-or-many.)

---

# 3) COMPONENT INVENTORY + CONNECTIONS

## 3a) System architecture components (#01)

**Browser Environment:** - *Extension UI Layer*: `Popup UI` (popup.html/popup.js â€" torrent list & controls), `Options Page` (options.html/options.js â€" server configuration), `Context Menu` (background.js â€" right-click actions) - *Extension Core Layer*: `Content Script` (content.js â€" DOM scanner & injector; drawio adds â€œTorrent detection UIâ€�), `Background Service Worker` (background.js â€" event handling & API proxy; drawio adds â€œAPI proxy & queueâ€�) - *Extension Library Layer*: `parser/` (magnet:// & .torrent parser), `scanner/` (DOM torrent detector), `api/BobaAPIClient` (HTTP client for services), `shared/` (constants & utilities) - *Extension Storage*: `Local Storage` (extension_config), `Session Storage` (discovered_torrents) - *Chrome APIs Used* (drawio-

only panel): chrome.runtime (messaging), chrome.tabs (tab management), chrome.tabGroups (group queries), chrome.contextMenus (right-click menu), chrome.storage (local & session), chrome.notifications (desktop alerts), fetch() API (HTTP requests), chrome.downloads (file downloads)

**Docker Host:** - *Boba Services Network*: Boba FastAPI (Port **7187** â€" metadata & search API; torrent info, search, health checks), Boba Go Service (Port **7189** â€" core torrent operations; add, remove, control) - *Torrent Network*: qBitTorrent WebUI (Port **8080** â€" torrent client API), qBitTorrent Database (resume data, settings, logs) - *Docker Networks* (drawio-only): boba-network (internal mesh: FastAPIâ†"ï¸ŽGoâ†"ï¸ŽqBit), bridge/host (external port mapping 7187, 7189, 8080), Volume Mounts (/downloads, /config)

**External Services:** BitTorrent Trackers (DHT, PEX, UDP announce; peer discovery & metadata; magnet resolution), Web Pages / Indexers (torrent sites & search engines; magnet hosting; .torrent download pages)

**Connections (#01):** - Popup â†"ï¸Ž BackgroundSW : chrome.runtime.sendMessage (bidirectional) - Options â†"ï¸Ž LocalStorage : Save/Load config - ContextMenu â†' BackgroundSW : Trigger action - ContentScript â†"ï¸Ž BackgroundSW : chrome.tabs.sendMessage (bidirectional) - ContentScript â†' Scanner : Scan DOM - Scanner â†' Parser : Parse links - BackgroundSW â†' BobaAPIClient : HTTP requests - BobaAPIClient â†' Shared : API calls - ContentScript â†' SessionStorage : Store discovered - ContentScript â†' WebPages : Inject detection UI - BobaAPIClient â†' FastAPI : REST API :7187 - BobaAPIClient â†' GoService : REST API :7189 - FastAPI â†' qBitTorrent : Proxy operations - GoService â†' qBitTorrent : Direct API - qBitTorrent â†' qBitDB : Persist state - qBitTorrent â†' Trackers : Announce/Scrape - (drawio) WebPages â†' ContentScript : dashed (injection inbound)

## 3b) Module/file component diagram (#06) â€" code organization

8 packages, each with files + key functions (from .mmd): - **parser/**: magnetParser.js (parseMagnetURI(), extractInfoHash()), torrentParser.js (parseTorrentFile(), extractMetadata()), hashNormalizer.js (normalizeInfoHash(), validateHash()) - **scanner/**: domScanner.js (scanForLinks(), MutationObserver), linkExtractor.js (extractMagnetLinks(), extractTorrentLinks()), torrentBadge.js (injectBadge(), highlightTorrents()) - **api/**: BobaAPIClient.js (sendTorrent(), sendBatch(), healthCheck()), authManager.js (getAuthHeaders(), refreshToken()), retryHandler.js (exponentialBackoff(), circuitBreaker()) - **background/**: background.js (event listeners, message router), contextMenuHandler.js (createMenus(), handleClicks()), queueManager.js (enqueue(), processQueue(), retryFailed()), tabGroupProcessor.js (queryGroups(), processTabs()) - **content/**: content.js (initialize(), DOM scan trigger), contentUI.js (injectPanel(), showTorrentList()) - **popup/**: popup.js (renderTorrentList(), handleSend(), handleConfig()), popup.html, popup.css - **options/**: options.js (saveConfig(), loadConfig(), validateServer()), options.html - **shared/**: constants.js (MSG_TYPES, API_ENDPOINTS, DEFAULTS), utils.js (debounce(), throttle(), formatBytes()), storageAPI.js (getConfig(), setConfig(), getHistory())

**Module dependencies (#06, both mmd+puml agree):** - scanner â†' parser (uses) - content â†' scanner (uses) - content â†' background (messaging, dashed) - popup â†' background (messaging, dashed) - background â†' api (uses) - background â†' shared (uses) - popup â†' shared (uses) -

options â†’ shared (uses) - api â†’ shared (uses) - parser â†’ shared (uses) - (mmd-only extra, likely typo) content â†’ content (uses)

---

# 4) END-TO-END SEQUENCES

## 4a) Single Torrent Send (#04) â€" participants: User, Popup UI, Content Script (CS), Background SW (BG), BobaAPIClient (API), FastAPI :7187, Go Service :7189, qBitTorrent :8080

**Detection phase:** 1. User â†’ CS: Browse torrent site 2. CS â†’ CS: MutationObserver detects `magnet:` link in DOM 3. CS â†’ CS: `parser.normalizeURL()` 4. CS â†’ BG: `chrome.runtime.sendMessage {type:"DISCOVERED_TORRENTS", torrents}` 5. BG â†’ Popup: `chrome.runtime.sendMessage {type:"UPDATE_POPUP", torrents}` 6. Popup â†’ Popup: Render torrent list 7. Popup â€"> User: Show discovered torrents

**Send phase:** 8. User â†’ Popup: Click â€œSendâ€ button 9. Popup â†’ BG: `chrome.runtime.sendMessage {type:"SEND_TORRENT", torrent}` 10. BG â†’ API: `sendTorrent(torrent)` 11. API â†’ API: Build request with auth headers â€" **X-Api-Key** + **Basic Auth**

**Two paths (mmd `par` / puml `alt` â€" both backends exercised):** - *FastAPI path*: API â†’ FastAPI `POST /api/torrents/add` (JSON body) â†’ FastAPI validates torrent data â†’ FastAPI â†’ qBit `POST /api/v2/torrents/add` (form-data: urls, category) â†’ qBit adds to BitTorrent engine â†’ qBit â†’ FastAPI `200 OK + {hash}` â†’ FastAPI â†’ API `200 OK + {hash, status}` - *Go Service path*: API â†’ GoSvc `POST /api/torrents/add` (JSON body) â†’ GoSvc validate + metadata lookup â†’ GoSvc â†’ qBit `POST /api/v2/torrents/add` (form-data) â†’ qBit adds to engine â†’ qBit â†’ GoSvc `200 OK + {hash}` â†’ GoSvc â†’ API `200 OK + {hash, status, metadata}`

**Completion:** 12. API â€"> BG: Promise resolved `{success, hash, status}` 13. BG â†’ BG: Store in `send_history`; Update badge count 14. BG â†’ Popup: `chrome.runtime.sendMessage {type:"SEND_COMPLETE", result}` 15. Popup â€"> User: âœ" Notification â€œTorrent addedâ€ - Note (right of qBit): magnet links resolved via DHT for metadata.

## 4b) Tab Group Batch Send (#05) â€" participants: User, Context Menu, Background SW (BG), Tab Groups API (TGAPI = chrome.tabGroups), Tabs API (TabsAPI = chrome.tabs), Content Scripts (CS), BobaAPIClient (API), qBitTorrent :8080

1. User â†’ Menu: Right-click on Tab Group
2. Menu â€"> User: Show context menu â€œSend all torrents to qBitâ€
3. User â†’ Menu: Select batch send option
4. Menu â†’ BG: `contextMenus.onClicked {menuItemId, parentMenuItemId}`
5. BG â†’ TGAPI: `chrome.tabGroups.query {title: groupName}`
6. TGAPI â€"> BG: Return group objects[] with `groupId`
7. BG â†’ TabsAPI: `chrome.tabs.query {groupId: gid}`
8. TabsAPI â€"> BG: Return tabs[] with `tabId, url`

**Loop â€" for each tab in group:** 9. BG â†' CS: `chrome.scripting.executeScript {target:{tabId}, files:["content.js"]}` 10. CS â†' CS: Scan DOM for torrents 11. CS â†' CS: `parser.normalizeURLs()` 12. CS â€"> BG: Return `[{torrents: [...]}]` (message `RETURN_TORRENTS` per #03)

**Batch send:** 13. BG â†' BG: Collect all torrents[] â€" **Flatten & deduplicate** 14. BG â†' API: `sendBatch(allTorrents)` 15. API â†' API: Build batch request with auth headers 16. API â†' qBit: `POST /api/v2/torrents/add` (`urls=magnet1\nmagnet2\n...`) â€" note: mmd #05 hits qBit directly; data-flow #03 routes through `POST /api/torrents/batch` on Boba Services first 17. qBit â†' qBit: Process each torrent in parallel 18. qBit â€"> API: `200 OK + results[]` 19. API â€"> BG: `{added: N, failed: [...]}` 20. BG â†' BG: Store results in `send_history` 21. BG â€"> User: `chrome.notifications` â€œN torrents added successfullyâ€

**Tab-groups data-flow detail (#03):** message types `DISCOVERED_TORRENTS` / `RETURN_TORRENTS`; batch endpoint `POST /api/torrents/batch` (JSON array of magnets) â†' proxied to qBit `/api/v2/torrents/add` (multiple urls) â†' result `200 OK + results[]` â†' notification â€œX torrents addedâ€.

---

# 5) STATE MACHINES

## 5a) Offline-queue state machine (#09)

States: `Idle`, `Pending`, `Sending`, `Queued`, `Retry`, `Completed`, `Failed` (+ initial/final `[*]`).

Transitions (source â†' target : guard/trigger): - `[*]` â†' Idle : Extension starts - Idle â†' Pending : Torrent detected - Pending â†' Sending : Online detected - Pending â†' Queued : Offline detected - Pending â†' Retry : Send failed - Sending â†' Completed : 200 OK - Sending â†' Failed : 4xx/5xx error - Sending â†' Retry : Timeout - Queued â†' Pending : Back online - Queued â†' Failed : Max age exceeded - Retry â†' Sending : Retry attempt - Retry â†' Failed : Max retries exceeded - Retry â†' Queued : Still offline - Completed â†' `[*]` : Cleanup after TTL - Failed â†' Pending : User retries - Failed â†' `[*]` : Abandoned

State notes: - Pending: â€œTorrent detected in DOM / Awaiting send triggerâ€ - Sending: â€œHTTP request in flight / BobaAPIClient activeâ€ - Queued: â€œDevice offline / Stored for later retryâ€ - Retry: â€œExponential backoff 1s â†' 2s â†' 4s â†' 8sâ€ - Completed: â€œ200 OK received / Added to send_historyâ€ - Failed: â€œIrrecoverable error / Logged for reviewâ€

## 5b) Extension lifecycle (#08) â€" flow (not a formal state machine, a staged lifecycle)

Stages (in order): `Install` (from store or .crx) â†' `Grant` (permissions) â†' `Configure` (server settings) â†' `Discover` (auto-discover server) â†' `Browse` (torrent sites) â†' `Detect` (torrent links) â†' `Send` (to qBitTorrent) â†' `Monitor` (downloads) â†' loops back to Browse.

Stage transitions w/ guards: - Install â†' Grant : `chrome.runtime.onInstalled` - Grant â†' Configure : Permissions granted (storage, scripting, tabs) - Configure â†' Discover : Save settings `chrome.storage.local` - Discover â†' Browse : Health check passed / server reachable - Browse â†' Detect : DOM contains torrent links on page - Detect â†' Send : Torrents

discovered / shown in popup - Send â†’ Monitor : Send successful / 200 OK received - Monitor â†’ Browse : Continue browsing

Chrome events feeding lifecycle: - `chrome.runtime.onInstalled` â†’ Install - `chrome.runtime.onStartup` â†’ Browse (Restore config from storage) - `chrome.runtime.onUpdateAvailable` â†’ Configure (Show changelog / migrate data)

Per-stage user actions: A1 Grant host/storage/scripting permissions (Install+Grant); A2 Enter server URL, API key, qBit URL (Configure); A3 Test connection / auto-detect Docker (Discover); A4 Navigate to torrent sites (Browse); A5 Review detected torrents in popup (Detect); A6 Click send / batch send / auto-send (Send); A7 Check progress in popup & qBit WebUI (Monitor).

---

# 6) SECURITY CONTROLS + TRUST BOUNDARIES (#10)

Trust boundaries (subgraphs): **Browser Environment**, **Network Layer**, **Boba Services**, **qBitTorrent**.

**Browser Environment controls:** - `Extension Permissions` (manifest.json): host, storage, scripting, tabs, contextMenus - *Content Security Policy (CSP)*: `script-src: 'self'` (no inline scripts); `connect-src: *` (API endpoints only); `object-src: 'none'` (no plugins) - *Credential Storage*: `chrome.storage.local` â€" **AES-256-GCM encrypted**; `chrome.storage.session` â€" non-sensitive cache (plain) - *Crypto Operations*: `Offscreen Document` â€" Web Crypto API, `SubtleCrypto.encrypt`; `PBKDF2 Key Derivation` â€" user password â†’ key

**Network Layer controls:** `HTTPS Only` (TLS 1.2+ required); `Certificate Validation` (no self-signed certs / cert pinning)

**Boba Services controls:** `API Key Auth` (`X-Api-Key` header); `Basic Auth` (username:password); `Rate Limiting` (100 req/min per key)

**qBitTorrent controls:** `WebUI Security` (Cookie-based SID, CSRF protection); `IP Filtering` (allowlist mode)

**Control flow / boundary crossings (#10):** ExtPermissions â†’ CSP â†’ StorageSec â†’ Crypto â†’ (Encrypted payload) HTTPS â†’ (Mutual TLS) BobaServices; HTTPS â†’ (HTTPS + Cookie) qBitTorrent; BobaServices â†’ Auth â†’ RateLimit â†’ (Proxied request) qBitTorrent.

Additional security notes from #01 legend: HTTPS for all external comms | credentials in chrome.storage.local | offscreen document for crypto | retry queue for offline | rate limiting | CSP headers | permission validation for all API calls.

---

# 7) BUILD PIPELINE STAGES (#11)

Linear stages: **Code** â†’ **Lint** â†’ **Test** â†’ **Build** â†’ **Package** â†’ **Deployments**. - `Code`: Git repository, `src/` directory - `Lint`: ESLint + Prettier, style checks - `Test`: Jest unit tests, coverage > 80% - `Build`: Webpack/Vite, bundle + minify - `Package`: Extension ZIP, manifest validation

**Stage transitions (with gates):** - Code â†' Lint : git push - Lint â†' Test : Pass | Lint â†' Code : Fail - Test â†' Coverage : Pass - Coverage â†' Build : â‰¥ 80% | Coverage â†' Code : < 80% - Build â†' ManifestValid : Bundle OK - ManifestValid â†' Package : Valid - Package â†' ChromeStore : Auto-deploy - Package â†' AMO : Auto-deploy - Package â†' Opera : Manual - Package â†' Edge : Auto-deploy - Package â†' Yandex : Manual

**Deployment targets:** Chrome Web Store ($5 one-time fee, review 1â€“3 days); Firefox AMO (free, 1â€“7 days); Opera Addons (free, 2â€–5 days); Edge Addons (free, 1â€–5 days); Yandex Browser (free, 3â€–7 days).

**CI/CD â€" GitHub Actions triggers:** `on: push` / `on: pull_request` â†' Lint; `on: release published` â†' Build; `on: schedule` (weekly dependency check) â†' Test. > NOTE: contradicts Boba root constitution â€œNO CI/CD pipelines / no `.github/workflows`â€ â€" see open questions.

**Quality Gates:** Coverage â‰¥ 80%; 0 ESLint errors; Valid manifest.json; Package < 5MB.

---

# 8) COMPATIBILITY MATRIX (#12 â€" full, every cell)

Legend: âœ“ Full (native) | âš  Partial (polyfill/workaround) | âœ— No (not supported).

| Feature | Chrome | Firefox | Opera | Edge | Yandex |
|---|---|---|---|---|---|
| Core Torrent Detection | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full |
| Tab Group Batch Send | âœ“ Full | âš  Partial | âœ“ Full | âœ“ Full | âœ“ Full |
| Context Menu Actions | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full |
| Service Worker | âœ“ MV3 | âš  MV2 | âœ“ MV3 | âœ“ MV3 | âœ“ MV3 |
| Offscreen Document | âœ“ Full | âœ— N/A | âœ“ Full | âœ“ Full | âœ“ Full |
| chrome.storage.session | âœ“ Full | âœ— No | âœ“ Full | âœ“ Full | âœ“ Full |
| chrome.scripting API | âœ“ Full | âš  Polyfill | âœ“ Full | âœ“ Full | âœ“ Full |
| chrome.notifications | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full |
| Auto-Update | âœ“ Store | âœ“ Store | âœ“ Store | âœ“ Store | âš  Manual |
| Credential Encryption | âœ“ Full | âš  Fallback | âœ“ Full | âœ“ Full | âœ“ Full |
| Background Message | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full |
| Badge Counter | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full |
| Popup UI | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full |
| Options Page | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full |
| Content Script Injection | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full | âœ“ Full |

**Browser-specific notes:** - **Firefox:** Tab Groups API not available (uses tab query fallback). Offscreen Document not supported (uses background-page crypto via Web Crypto API). storage.session replaced with localStorage fallback. MV2 uses background page instead of service worker. scripting API uses `browser.tabs.executeScript` via webextension-polyfill. All core features work with polyfills. - **Yandex:** Requires Opera extension package format. Auto-update must be checked manually via Yandex extension store. All Chromium APIs fully supported. - **Opera/Edge:** Fully Chromium compatible. Edge uses same package as Chrome. Opera requires `manifest` key in package. - **Chrome:** Reference implementation platform. All MV3 APIs used natively. No polyfills required.

---

# 9) OPEN QUESTIONS / INCONSISTENCIES

1. **CI/CD contradiction:** #11 build pipeline mandates GitHub Actions (`on: push`, `on: release`, `on: schedule`) and â€œAuto-deployâ€ to stores. Boba root CLAUDE.md/Constitution Hard-Stop #1 forbids ALL CI/CD pipelines and `.github/workflows/*`. The plan MUST reconcile â€” likely the extension deploy pipeline runs as a manual script (`./ci.sh`-style), not GitHub Actions.
2. **Single-send routing ambiguity:** #04 single-send has the API client call BOTH FastAPI (`POST /api/torrents/add`) AND Go Service (`POST /api/torrents/add`) â€” modeled as parallel/alt. Is this an either-or (one configured backend) or genuinely dual-write? #02 shows FastAPI getting `POST /api/torrents/add` and Go getting `POST /api/torrents/batch` â€” inconsistent endpointâ†'service mapping vs #04.
3. **Batch endpoint path inconsistency:** #03/#02 route batch through Boba `POST /api/torrents/batch`; #05 sequence has BGâ†'APIâ†'qBit directly via `POST /api/v2/torrents/add`, skipping the Boba batch endpoint. Which is authoritative?
4. **Boba port reality vs diagrams:** diagrams use qBit WebUI port **8080**; the real Boba repo (per CLAUDE.md) uses qBittorrent on **7185** proxied via **7186**, merge service **7187**, boba-jackett **7189**. FastAPI=7187 and Go=7189 roughly align, but qBit 8080 does not match the repoâ€™s 7185/7186. Plan must map BobaLink endpoints onto real Boba services.
5. `extension_config` **â†"ï¸Ž** `discovered_torrents` **relationship** is labeled â€œreferences configâ€ but `discovered_torrents` has no `config_id` FK field â€" relationship is conceptual, not a declared FK. Confirm whether a FK column is needed.
6. `search_cache` **orphan:** no relationship; which service populates it (FastAPi search?) and where is it stored (chrome.storage? IndexedDB?) â€" storage backend for all 6 entities is unspecified (extension uses chrome.storage.local/session per #01; ER implies relational/SQL types). Need a decision: chrome.storage vs IndexedDB vs server-side DB.
7. **Auth model:** #04 says `X-Api-Key + Basic Auth` together; #10 lists API Key Auth and Basic Auth as separate controls. Confirm both are sent on every request vs per-service.
8. `source_tab_id`**,** `last_retry_at`**,** `last_health_check`**,** `magnet_uri`/`qbittorrent_hash`/`tags` **(send_history)** present in `.mmd` but dropped in `.drawio` â€" treat `.mmd` as authoritative (done above), but confirm the dropped fields are intended.
9. **contentâ†'content self-dependency (#06 mmd line 56)** is almost certainly a typo (should be contentâ†'shared or similar). Ignore unless intentional.
10. **MV2/MV3 dual-target:** matrix requires Firefox MV2 (background page) + Chromium MV3 (service worker). Build pipeline must produce two manifest variants + webextension-polyfill â€" not shown as a build stage in #11.